

Machine Learning - Python NumPy

July 8, 2026

1 PAO25_25_06_Python_NumPy



Autor: Elvis Pachacama **Recopilado por:** Pablo Aguilar * Repositorio: GitHub

1.1 PAO25-25 - NumPy

1.2 1. NumPy ndarray

- NumPy (**N**umerical **P**ython) es la librería estándar de facto para análisis numérico en Python.
- Estructura de datos multidimensional eficiente (escrita en C): **ndarray**.
- Colección de funciones para álgebra lineal y estadística descriptiva.
- Ayuda al procesamiento de datos (**np.where**).

```
[1]: #importar libreria
import numpy as np

# Línea 1: comentario descriptivo (no ejecutable).
# Línea 2: se importa la librería numpy y se le asigna el alias 'np', que es la
#          convención estándar de la comunidad para referenciarla de forma
#          ↪ corta.
# Resultado esperado: no hay salida visible; numpy queda disponible como 'np'.
```

1.2.1 1.1 Performance Sample, NumPy vs Plain Python

```
[2]: my_list = list(range(10000000)) # Utilizando Listas Python
      %time my_list = [x * 2 for x in my_list]

# Línea 1: se crea una lista Python de 10 millones de enteros (0..9999999) con
      ↳ range() convertido a list.
# Línea 2: %time es una 'magic function' de Jupyter que mide el tiempo de
      ↳ ejecución de la
#           instrucción; aquí recorre la lista con una list comprehension
      ↳ multiplicando cada
#           elemento por 2 y reasigna el resultado a my_list.
# Resultado esperado: se imprime el tiempo de CPU/wall transcurrido (p. ej.
      ↳ "Wall time: 900 ms"
#           aprox., el valor exacto depende del hardware); no hay impresión de
      ↳ los valores.
```

CPU times: total: 562 ms

Wall time: 550 ms

```
[3]: my_arr = np.arange(10000000) # Utilizando NumPy Arrays
      %time my_arr2 = my_arr * 2

# Línea 1: np.arange genera un ndarray de 10 millones de enteros (equivalente
      ↳ vectorizado a range).
# Línea 2: %time mide cuánto tarda la multiplicación vectorizada my_arr * 2,
      ↳ que NumPy ejecuta
#           en código C compilado en lugar de un bucle interpretado por Python.
# Resultado esperado: el tiempo reportado es notablemente menor que el de la
      ↳ lista Python
#           (normalmente uno o dos órdenes de magnitud más rápido), demostrando
      ↳ la eficiencia
#           de las operaciones vectorizadas de NumPy.
```

CPU times: total: 15.6 ms

Wall time: 12.5 ms

1.2.2 1.2 Formas de crear ndarrays

```
[4]: array = np.array([ [2,71,0,34], [2,171,-35,34] ]) # 2D
      print(array)

# Línea 1: np.array crea un ndarray bidimensional (2 filas x 4 columnas) a
      ↳ partir de una lista
#           de listas.
# Línea 2: print muestra el arreglo con formato de matriz.
# Resultado esperado:
# [[ 2  71  0  34]
```

```
# [ 2 171 -35 34]]
```

```
[[ 2 71 0 34]
 [ 2 171 -35 34]]
```

```
[5]: shopping_list = [ ['onions', 'carrots', 'celery'], ['apples', 'oranges', 'grapes']] # 2D
array = np.array(shopping_list) # convertir una lista en una ndarray
print(array)
print(type(array))

# Línea 1: se define una lista de listas de strings (2 filas x 3 columnas).
# Línea 2: np.array convierte esa lista de listas en un ndarray bidimensional de tipo string.
# Línea 3: se imprime el contenido del ndarray.
# Línea 4: type(array) muestra la clase del objeto.
# Resultado esperado:
# [['onions' 'carrots' 'celery']
#  ['apples' 'oranges' 'grapes']]
# <class 'numpy.ndarray'>
```

```
[['onions' 'carrots' 'celery']
 ['apples' 'oranges' 'grapes']]
<class 'numpy.ndarray'>
```

```
[6]: print(np.array([100,10,1])) # array a partir de una lista
print(np.arange(2,10,2.1)) # array a partir de una secuencia(range) 1D

# Línea 1: crea un ndarray 1D a partir de una lista literal de enteros y lo imprime.
# Línea 2: np.arange(2,10,2.1) genera valores desde 2 (inclusive) hasta 10 (exclusive) con
#         paso 2.1; al usar un paso float, el resultado es un array de floats.
# Resultado esperado:
# [100 10 1]
# [2. 4.1 6.2 8.3]
```

```
[100 10 1]
[2. 4.1 6.2 8.3]
```

```
[7]: # Generar arrays con valores fijos
print(np.zeros(2)) # 1D
print(np.ones((4,3))) # matrix 2x2
print(np.empty((2,2,2))) # matrix 2x2x2, no inicializados
print(np.full((5,2),2)) # matrix 5x2
print(np.eye(3)) # matrix identity

# Línea 1: comentario descriptivo de la sección.
```

```

# Línea 2: np.zeros(2) crea un vector 1D de 2 ceros: [0. 0.]
# Línea 3: np.ones((4,3)) crea una matriz 4x3 llena de unos.
# Línea 4: np.empty((2,2,2)) reserva memoria para un array 2x2x2 SIN
    ↪ inicializar sus valores
#
    (los valores mostrados son 'basura' de memoria, pueden variar en
    ↪ cada ejecución).
# Línea 5: np.full((5,2),2) crea una matriz 5x2 rellena con el valor 2.
# Línea 6: np.eye(3) crea la matriz identidad 3x3 (unos en la diagonal, ceros
    ↪ en el resto).
# Resultado esperado: cinco impresiones sucesivas de arrays con las formas y
    ↪ valores descritos
#
    (el contenido de np.empty es indeterminado).

```

```

[0. 0.]
[[1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]]
[[[9.88e-324 3.51e-322]
  [0.00e+000 1.68e-322]]

 [[9.88e-324 8.45e-322]
  [      nan 1.68e-322]]]
[[2 2]
 [2 2]
 [2 2]
 [2 2]
 [2 2]]
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]

```

1.2.3 1.3 Obtener información sobre ndarray

```

[8]: my_ndarray = np.array([2,71,0,34])
     print(my_ndarray)

# Línea 1: crea un ndarray 1D con 4 elementos.
# Línea 2: lo imprime.
# Resultado esperado: [ 2 71  0 34]

```

```

[ 2 71  0 34]

```

```

[9]: print(my_ndarray.argmax()) # índice del valor más alto
     print(np.amax(my_ndarray))
     print(my_ndarray.argmin()) # índice del valor más bajo
     print(np.amin(my_ndarray))
     print(my_ndarray.nonzero()) # retorna los índices de los elementos distintos a 0

```

```

# Línea 1: argmax() devuelve el índice (posición) donde se encuentra el valor
↳ máximo (71 -> índice 1).
# Línea 2: np.amax devuelve directamente el valor máximo (71).
# Línea 3: argmin() devuelve el índice del valor mínimo (0 -> índice 2).
# Línea 4: np.amin devuelve directamente el valor mínimo (0).
# Línea 5: nonzero() devuelve, en una tupla, los índices de los elementos
↳ distintos de cero.
# Resultado esperado:
# 1
# 71
# 2
# 0
# (array([0, 1, 3]),)

```

```

1
71
2
0
(array([0, 1, 3]),)

```

```

[10]: my_ndarray = np.full((5, 4, 2), 2) # matrix 5x2x2
print(my_ndarray)
print(my_ndarray.size) # número de elementos
print(my_ndarray.shape) # dimensiones
print(my_ndarray.ndim) # número de dimensiones

# Línea 1: crea un array 3D de forma (5,4,2) relleno con el valor 2.
# Línea 2: imprime el contenido completo del array.
# Línea 3: .size da el número total de elementos (5*4*2 = 40).
# Línea 4: .shape da la tupla de dimensiones (5, 4, 2).
# Línea 5: .ndim da el número de ejes/dimensiones (3).
# Resultado esperado: se imprime el array de 5 bloques de 4x2 llenos de 2,
↳ luego:
# 40
# (5, 4, 2)
# 3

```

```

[[[2 2]
  [2 2]
  [2 2]
  [2 2]]

```

```

[[2 2]
 [2 2]
 [2 2]
 [2 2]]

```

```

[[2 2]
 [2 2]
 [2 2]
 [2 2]]

[[2 2]
 [2 2]
 [2 2]
 [2 2]]

[[2 2]
 [2 2]
 [2 2]
 [2 2]]]
40
(5, 4, 2)
3

```

```

[11]: my_ndarray = np.array([2,71,0,34])
print(my_ndarray.size) # número de elementos
print(my_ndarray.shape) # dimensiones
print(my_ndarray.ndim) # número de dimensiones

# Línea 1: se redefine my_ndarray como un array 1D de 4 elementos.
# Línea 2: .size = número total de elementos = 4.
# Línea 3: .shape = (4,) porque es un array de una sola dimensión con 4
↪ elementos.
# Línea 4: .ndim = 1 porque solo tiene un eje.
# Resultado esperado:
# 4
# (4,)
# 1

```

```

4
(4,)
1

```

Redimensionar los datos

```

[12]: my_ndarray = np.array([[0, 71, 21, 19, 213, 412, 111, 98]]) # matrix 1x8
print(my_ndarray)
print(my_ndarray.shape)
print(my_ndarray.ndim)

# Línea 1: se crea un array 2D de forma (1,8) porque los datos están dentro de
↪ doble corchete
#          (una lista que contiene una única lista de 8 elementos).
# Línea 2: imprime el array.

```

```

# Línea 3: shape = (1, 8), una fila y ocho columnas.
# Línea 4: ndim = 2, porque aunque tenga una sola fila sigue siendo
↳bidimensional.
# Resultado esperado:
# [[ 0  71  21  19 213 412 111  98]]
# (1, 8)
# 2

```

```

[[ 0  71  21  19 213 412 111  98]]
(1, 8)
2

```

```

[13]: new_dims = my_ndarray.reshape(2, 4) # cambiar las dimensiones a 2x4
print(new_dims)
print(new_dims.ndim)

# Línea 1: reshape reorganiza los 8 elementos existentes en una matriz de 2
↳filas x 4 columnas
# (el número total de elementos debe mantenerse: 2*4 = 8).
# Línea 2: imprime la nueva matriz.
# Línea 3: ndim = 2.
# Resultado esperado:
# [[ 0  71  21  19]
# [213 412 111  98]]
# 2

```

```

[[ 0  71  21  19]
 [213 412 111  98]]
2

```

```

[14]: new_dims = my_ndarray.reshape(4, 2) # cambiar las dimensiones a 4x2
print(new_dims)
print(new_dims.ndim)

# Línea 1: se reorganizan los mismos 8 elementos en 4 filas x 2 columnas.
# Línea 2: imprime la matriz resultante.
# Línea 3: ndim = 2.
# Resultado esperado:
# [[ 0  71]
# [ 21  19]
# [213 412]
# [111  98]]
# 2

```

```

[[ 0  71]
 [ 21  19]
 [213 412]
 [111  98]]

```

```

2

```

```
[15]: x = np.array([100, 10, 1]).reshape(3,1)
print(x.shape)
print(x)

# Línea 1: crea un array 1D de 3 elementos y lo convierte (reshape) en un
↪vector columna 3x1.
# Línea 2: imprime la forma (3, 1).
# Línea 3: imprime el vector columna.
# Resultado esperado:
# (3, 1)
# [[100]
#  [ 10]
#  [  1]]
```

(3, 1)
[[100]
[10]
[1]]

```
[16]: # reducir a 1D
new_dims = np.arange(9).reshape(3,3)
print(new_dims)
print(new_dims.flatten())

# Línea 1: comentario descriptivo.
# Línea 2: crea un array de 0 a 8 y lo reorganiza en una matriz 3x3.
# Línea 3: imprime la matriz.
# Línea 4: flatten() devuelve una copia 1D con todos los elementos en orden
↪(row-major por defecto).
# Resultado esperado:
# [[0 1 2]
#  [3 4 5]
#  [6 7 8]]
# [0 1 2 3 4 5 6 7 8]
```

[[0 1 2]
[3 4 5]
[6 7 8]]
[0 1 2 3 4 5 6 7 8]

```
[17]: # reducir
new_dims = np.arange(16).reshape(4,4)
print(new_dims)
print(new_dims.reshape(2,-1)) # el segundo valor se infiere del primero

# Línea 1: comentario descriptivo.
# Línea 2: crea un array de 0 a 15 y lo reorganiza en una matriz 4x4.
# Línea 3: imprime la matriz 4x4.
```



```
# Línea 4: reshape(2,-1) pide 2 filas y usa -1 para que NumPy calcule
    ↪ automáticamente el número
#           de columnas necesario (16 elementos / 2 filas = 8 columnas).
# Resultado esperado:
# [[ 0  1  2  3]
#  [ 4  5  6  7]
#  [ 8  9 10 11]
#  [12 13 14 15]]
# [[ 0  1  2  3  4  5  6  7]
#  [ 8  9 10 11 12 13 14 15]]
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]]
[[ 0  1  2  3  4  5  6  7]
 [ 8  9 10 11 12 13 14 15]]
```

1.2.4 1.4 Manipulación de ndarrays

```
[18]: # Cambiar ejes
new_dims = np.arange(16).reshape(8,2)
print(new_dims)
print(new_dims.swapaxes(0,1))

# Línea 1: comentario descriptivo.
# Línea 2: crea un array de 0 a 15 reorganizado en 8 filas x 2 columnas.
# Línea 3: imprime la matriz 8x2.
# Línea 4: swapaxes(0,1) intercambia el eje 0 (filas) con el eje 1 (columnas),
    ↪ obteniendo la
#           transpuesta de la matriz (2 filas x 8 columnas).
# Resultado esperado: matriz 8x2 original, seguida de su transpuesta:
# [[ 0  2  4  6  8 10 12 14]
#  [ 1  3  5  7  9 11 13 15]]
```

```
[[ 0  1]
 [ 2  3]
 [ 4  5]
 [ 6  7]
 [ 8  9]
 [10 11]
 [12 13]
 [14 15]]
[[ 0  2  4  6  8 10 12 14]
 [ 1  3  5  7  9 11 13 15]]
```

```
[19]: # Flip
new_dims = np.arange(16).reshape(4,4)
```

```

print(new_dims)
print(np.flip(new_dims, axis = None))

# Línea 1: comentario descriptivo.
# Línea 2: crea la matriz 4x4 de 0 a 15.
# Línea 3: la imprime.
# Línea 4: np.flip con axis=None invierte el orden de TODOS los elementos
    ↳(equivale a invertir
#           en todos los ejes a la vez), no solo filas o columnas por separado.
# Resultado esperado:
# [[15 14 13 12]
#  [11 10  9  8]
#  [ 7  6  5  4]
#  [ 3  2  1  0]]

```

```

[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]]
[[15 14 13 12]
 [11 10  9  8]
 [ 7  6  5  4]
 [ 3  2  1  0]]

```

```

[20]: # ordenar
array1 = np.array([10,2,9,17])
array1.sort()
print(array1)

# Línea 1: comentario descriptivo.
# Línea 2: crea un array 1D desordenado.
# Línea 3: sort() ordena el array IN PLACE (modifica el propio array) de menor
    ↳a mayor.
# Línea 4: imprime el array ya ordenado.
# Resultado esperado: [ 2  9 10 17]

```

```

[ 2  9 10 17]

```

```

[21]: # Juntar dos arrays
a1 = [0,0,0,0,0,0]
a2 = [1,1,1,1,1,1]
print(np.hstack((a1,a2))) # una al lado de la otra
print(np.hstack((a1,a2)).shape)
print(np.vstack((a1,a2))) # una encima de la otra
print(np.vstack((a1,a2)).shape)

# Línea 1: comentario descriptivo.
# Línea 2-3: se definen dos listas Python de 6 elementos cada una.

```

```

# Línea 4: hstack concatena horizontalmente (apilado en un único vector de 12
↪ elementos).
# Línea 5: shape del resultado de hstack: (12,).
# Línea 6: vstack apila verticalmente (crea una matriz de 2 filas x 6 columnas).
# Línea 7: shape del resultado de vstack: (2, 6).
# Resultado esperado:
# [0 0 0 0 0 0 1 1 1 1 1 1]
# (12,)
# [[0 0 0 0 0 0]
#  [1 1 1 1 1 1]]
# (2, 6)

```

```

[0 0 0 0 0 0 1 1 1 1 1 1]
(12,)
[[0 0 0 0 0 0]
 [1 1 1 1 1 1]]
(2, 6)

```

```

[22]: array1 = np.array([[2, 4], [6, 8]])
      array2 = np.array([[3, 5], [7, 9]])
      print(np.concatenate((array1, array2), axis = 0))

```

```

# Línea 1-2: se crean dos matrices 2x2.
# Línea 3: np.concatenate con axis=0 une las matrices apilando filas (una
↪ debajo de la otra),
#           resultando en una matriz de 4 filas x 2 columnas.
# Resultado esperado:
# [[2 4]
#  [6 8]
#  [3 5]
#  [7 9]]

```

```

[[2 4]
 [6 8]
 [3 5]
 [7 9]]

```

```

[23]: # dividir un array
      array = np.arange(16).reshape(4,4)
      print(array)
      print(np.array_split(array, 2, axis = 0)) # divide array en 2 partes iguales
      print(np.array_split(array, 3)) # divide array en 3 partes "iguales"

# Línea 1: comentario descriptivo.
# Línea 2: crea la matriz 4x4 de 0 a 15.
# Línea 3: la imprime.
# Línea 4: array_split(array, 2, axis=0) divide la matriz en 2 partes iguales
↪ por filas (axis 0),

```

```
#           devolviendo una lista de 2 sub-arrays de 2x4 cada uno.
# Línea 5: array_split(array, 3) (axis=0 por defecto) intenta dividir en 3
↳ partes; como 4 filas
#           no es divisible exactamente entre 3, reparte de forma lo más pareja
↳ posible
#           (algunas partes tendrán 2 filas y otras 1).
# Resultado esperado: la matriz original, luego una lista con 2 arrays de forma
↳ (2,4), luego una
#           lista con 3 arrays de tamaños desiguales (2,4), (1,4), (1,4).
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]]
[array([[0, 1, 2, 3],
        [4, 5, 6, 7]]), array([[ 8,  9, 10, 11],
                               [12, 13, 14, 15]])]
[array([[0, 1, 2, 3],
        [4, 5, 6, 7]]), array([[ 8,  9, 10, 11]]), array([[12, 13, 14, 15]])]
```

```
[24]: # dividir un array
array = np.arange(16).reshape(4,4)
print(array)
print(np.array_split(array, [3], axis = 1)) # divide array, por la fila 3

# Línea 1: comentario descriptivo.
# Línea 2: crea de nuevo la matriz 4x4.
# Línea 3: la imprime.
# Línea 4: al pasar una lista de índices [3] en lugar de un entero, array_split
↳ corta la matriz
#           EN la columna de índice 3 (axis=1 = columnas): la primera parte
↳ tendrá las columnas
#           0,1,2 y la segunda parte la columna 3.
# Resultado esperado: lista con dos arrays: uno de forma (4,3) y otro de forma
↳ (4,1).
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]]
[array([[ 0,  1,  2],
        [ 4,  5,  6],
        [ 8,  9, 10],
        [12, 13, 14]]), array([[ 3],
                               [ 7],
                               [11],
                               [15]])]
```

```
[25]: split = np.array_split(array, [3], axis = 1)
print(split[0][3][0])

# Línea 1: se repite la división por columnas en el índice 3, guardando el
↳ resultado en 'split'
#           (lista de 2 sub-arrays).
# Línea 2: split[0] es el primer sub-array (columnas 0,1,2); [3] accede a su
↳ fila de índice 3
#           (la última fila); [0] accede al primer elemento de esa fila.
# Resultado esperado: 12 (el elemento en la fila 3, columna 0 de la matriz
↳ original).
```

12

```
[26]: array = np.arange(64).reshape(8,8)
print(array)
print(np.array_split(array, [1, 3], axis = 0)) # divide array por la fila 1 y 3
print(np.array_split(array, [1, 3], axis = 1)) # divide array por la columna 1
↳ y 3
# axis = n_dimensions - 1

# Línea 1: crea un array de 0 a 63 reorganizado en una matriz 8x8.
# Línea 2: la imprime.
# Línea 3: divide por filas (axis=0) en los cortes de índice 1 y 3, generando 3
↳ sub-matrices:
#           filas [0], filas [1,2], filas [3..7].
# Línea 4: divide por columnas (axis=1) en los mismos cortes 1 y 3, generando 3
↳ sub-matrices
#           por columnas en vez de filas.
# Línea 5: comentario recordando que axis = n dimensiones - 1 se refiere al
↳ último eje (columnas
#           en un array 2D).
# Resultado esperado: la matriz 8x8 completa, seguida de la lista de 3
↳ sub-arrays divididos por
#           fila y luego la lista de 3 sub-arrays divididos por columna.
```

```
[[ 0  1  2  3  4  5  6  7]
 [ 8  9 10 11 12 13 14 15]
 [16 17 18 19 20 21 22 23]
 [24 25 26 27 28 29 30 31]
 [32 33 34 35 36 37 38 39]
 [40 41 42 43 44 45 46 47]
 [48 49 50 51 52 53 54 55]
 [56 57 58 59 60 61 62 63]]
[array([[0, 1, 2, 3, 4, 5, 6, 7]]), array([[ 8,  9, 10, 11, 12, 13, 14, 15],
      [16, 17, 18, 19, 20, 21, 22, 23]]), array([[24, 25, 26, 27, 28, 29, 30,
31],
      [32, 33, 34, 35, 36, 37, 38, 39],
```

```

[40, 41, 42, 43, 44, 45, 46, 47],
[48, 49, 50, 51, 52, 53, 54, 55],
[56, 57, 58, 59, 60, 61, 62, 63]])]
[array([[ 0],
       [ 8],
       [16],
       [24],
       [32],
       [40],
       [48],
       [56]]), array([[ 1,  2],
                      [ 9, 10],
                      [17, 18],
                      [25, 26],
                      [33, 34],
                      [41, 42],
                      [49, 50],
                      [57, 58]])], array([[ 3,  4,  5,  6,  7],
                      [11, 12, 13, 14, 15],
                      [19, 20, 21, 22, 23],
                      [27, 28, 29, 30, 31],
                      [35, 36, 37, 38, 39],
                      [43, 44, 45, 46, 47],
                      [51, 52, 53, 54, 55],
                      [59, 60, 61, 62, 63]])])

```

1.3 2. Índices y slicing

- De manera análoga a índices y slicing para listas.

```

[27]: array = np.arange(10, 16)
print(array)
print(array[-1]) # acceso unidimensional
print(array[:-1]) # slice

# Línea 1: crea un array 1D con valores de 10 a 15.
# Línea 2: lo imprime.
# Línea 3: array[-1] accede al último elemento usando índice negativo (15).
# Línea 4: array[:-1] es un slice desde el inicio hasta (sin incluir) el último
↪ elemento.
# Resultado esperado:
# [10 11 12 13 14 15]
# 15
# [10 11 12 13 14]

```

```
[10 11 12 13 14 15]
```

```
15
```

```
[10 11 12 13 14]
```

```
[28]: # bidimensional
array = np.arange(16).reshape(4,4)
print(array)
print(array[2, 2]) # fila, columna
print(array[3][2]) # fila, columna

# Línea 1: comentario descriptivo.
# Línea 2: crea la matriz 4x4 de 0 a 15.
# Línea 3: la imprime.
# Línea 4: array[2,2] accede al elemento en fila 2, columna 2 usando la
↳ notación NumPy (más
# eficiente, un único acceso).
# Línea 5: array[3][2] hace lo mismo (fila 3, columna 2) pero mediante doble
↳ indexación
# encadenada (primero obtiene la fila 3 como sub-array, luego su
↳ elemento 2).
# Resultado esperado:
# 10
# 14
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]]
10
14
```

```
[29]: # multidimensional
array = np.arange(36).reshape(2,3,6)
print(array)
print(array[0,2,4])
print(array[1][2][4])

# Línea 1: comentario descriptivo.
# Línea 2: crea un array 3D de forma (2,3,6) con valores de 0 a 35.
# Línea 3: lo imprime (2 bloques de matrices 3x6).
# Línea 4: array[0,2,4] accede al bloque 0, fila 2, columna 4.
# Línea 5: array[1][2][4] accede al bloque 1, fila 2, columna 4 mediante
↳ indexación encadenada.
# Resultado esperado:
# 16
# 34
```

```
[[[ 0  1  2  3  4  5]
 [ 6  7  8  9 10 11]
 [12 13 14 15 16 17]]

 [[18 19 20 21 22 23]]
```

```
[24 25 26 27 28 29]
[30 31 32 33 34 35]]]
16
34
```

NumPy es row major

```
[30]: row_major = np.array([1, 2, 3, 4, 5, 6])
row_major = row_major.reshape(2,3) # by default, row major
print(row_major)

# Línea 1: crea un array 1D de 6 elementos.
# Línea 2: reshape(2,3) reorganiza relleno primero por filas (orden 'C' /
↳row-major, que es
#           el orden por defecto en NumPy).
# Línea 3: imprime la matriz resultante.
# Resultado esperado:
# [[1 2 3]
#   [4 5 6]]
```

```
[[1 2 3]
 [4 5 6]]
```

```
[31]: row_major = np.array([1, 2, 3, 4, 5, 6])
row_major = row_major.reshape(2,3,order='C') # by default, row major
print(row_major)

col_major = np.array([1, 2, 3, 4, 5, 6])
col_major = col_major.reshape(2,3,order='F') # col major
print(col_major)

# Línea 1-2: se crea un array y se le aplica reshape con order='C' (orden C /
↳row-major
#           explícito), llenando la matriz fila por fila.
# Línea 3: imprime el resultado (idéntico al reshape por defecto).
# Línea 4-5: se crea otro array y se aplica reshape con order='F' (Fortran /
↳column-major),
#           que llena la matriz columna por columna en lugar de fila por fila.
# Línea 6: imprime el resultado con orden Fortran.
# Resultado esperado:
# [[1 2 3]
#   [4 5 6]]
# [[1 3 5]
#   [2 4 6]]
```

```
[[1 2 3]
 [4 5 6]]
[[1 3 5]
 [2 4 6]]
```


Slice

```
[32]: # se puede hacer slice por fila o columna
array = np.arange(81).reshape(9,9)
print(array)
print(array[1:3, 2::2])

# Línea 1: comentario descriptivo.
# Línea 2: crea una matriz 9x9 con valores de 0 a 80.
# Línea 3: la imprime.
# Línea 4: array[1:3, 2::2] selecciona las filas 1 y 2 (índice 3 excluido), y
    ↪ dentro de esas
#         filas, las columnas desde el índice 2 hasta el final tomando de 2 en
    ↪ 2 (columnas 2,4,6,8).
# Resultado esperado:
# [[11 13 15 17]
#  [20 22 24 26]]
```

```
[[ 0  1  2  3  4  5  6  7  8]
 [ 9 10 11 12 13 14 15 16 17]
 [18 19 20 21 22 23 24 25 26]
 [27 28 29 30 31 32 33 34 35]
 [36 37 38 39 40 41 42 43 44]
 [45 46 47 48 49 50 51 52 53]
 [54 55 56 57 58 59 60 61 62]
 [63 64 65 66 67 68 69 70 71]
 [72 73 74 75 76 77 78 79 80]]
[[11 13 15 17]
 [20 22 24 26]]
```

```
[33]: # se puede hacer slice por fila o columna
array = np.arange(9).reshape(3,3)
print(array)
print(array[0,:])

# Línea 1: comentario descriptivo.
# Línea 2: crea una matriz 3x3 con valores de 0 a 8.
# Línea 3: la imprime.
# Línea 4: array[0,:] selecciona toda la fila de índice 0 (todas las columnas).
# Resultado esperado:
# [[0 1 2]
#  [3 4 5]
#  [6 7 8]]
# [0 1 2]
```

```
[[0 1 2]
 [3 4 5]
 [6 7 8]]
```

[0 1 2]

Diferencias entre índices y slicing en listas y ndarrays:

- ¡Retornan una referencia, no una copia!

```
[34]: # uso de slice en listas, retorna copia
lista = [0,1,2,3,4,5,6,7,8,9]
my_copy = lista[0:2] # devuelve una copia
print(my_copy)
my_copy[0] = 222
print(lista)
print(my_copy)

# Línea 1: comentario descriptivo.
# Línea 2: se define una lista Python de 10 elementos.
# Línea 3: lista[0:2] crea una COPIA independiente con los dos primeros
↪ elementos [0,1].
# Línea 4: imprime la copia.
# Línea 5: se modifica el primer elemento de la copia (no afecta a la lista
↪ original).
# Línea 6: imprime la lista original (sin cambios).
# Línea 7: imprime la copia modificada.
# Resultado esperado:
# [0, 1]
# [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
# [222, 1]
```

[0, 1]

[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

[222, 1]

```
[35]: # slicing retorna una referencia en ndarrays
array = np.arange(10)
my_copy = array[2:5] # devuelve una referencia
print(array)
print(my_copy)
my_copy[0] = 222
print(array)
print(my_copy)

# Línea 1: comentario descriptivo.
# Línea 2: crea un array 1D de 0 a 9.
# Línea 3: array[2:5] NO crea una copia, devuelve una VISTA (referencia a la
↪ misma memoria).
# Línea 4: imprime el array original.
# Línea 5: imprime la vista (elementos 2,3,4).
# Línea 6: modifica el primer elemento de la vista; como comparte memoria con
↪ 'array', el
```

```

#          array original también se modifica.
# Línea 7: imprime el array original (YA modificado, a diferencia de las
↳ listas).
# Línea 8: imprime la vista modificada.
# Resultado esperado:
# [0 1 2 3 4 5 6 7 8 9]
# [2 3 4]
# [ 0  1 222  3  4  5  6  7  8  9]
# [222  3  4]

```

```

[0 1 2 3 4 5 6 7 8 9]
[2 3 4]
[ 0  1 222  3  4  5  6  7  8  9]
[222  3  4]

```

```

[36]: # copiar ndarrays
array = np.arange(10)
my_copy = array[2:5].copy() # hay que hacer la copia explícitamente
my_copy[0] = 222
print(array)
print(my_copy)

# Línea 1: comentario descriptivo.
# Línea 2: crea un array 1D de 0 a 9.
# Línea 3: al añadir .copy() se obliga a NumPy a crear una copia independiente
↳ de memoria en
#          lugar de una vista.
# Línea 4: se modifica el primer elemento de la copia.
# Línea 5: imprime el array original (SIN cambios, porque ahora sí es
↳ independiente).
# Línea 6: imprime la copia modificada.
# Resultado esperado:
# [0 1 2 3 4 5 6 7 8 9]
# [222  3  4]

```

```

[0 1 2 3 4 5 6 7 8 9]
[222  3  4]

```

- Permite asignar valores a un corte

```

[37]: array = np.arange(10)
print(array)
print(array[0:3])
array[0:3] = -1
print(array)

# Línea 1: crea un array 1D de 0 a 9.
# Línea 2: lo imprime.

```

```

# Línea 3: imprime el slice de los 3 primeros elementos.
# Línea 4: asigna el valor -1 a todo el slice [0:3] (broadcasting de un escalar
↳ a un rango).
# Línea 5: imprime el array modificado.
# Resultado esperado:
# [0 1 2 3 4 5 6 7 8 9]
# [0 1 2]
# [-1 -1 -1 3 4 5 6 7 8 9]

```

```

[0 1 2 3 4 5 6 7 8 9]
[0 1 2]
[-1 -1 -1 3 4 5 6 7 8 9]

```

- Slicing condicionales (mask)

```

[38]: # slicing condicional
array = np.arange(-3,4)
print(array)
mask = array %2 == 0
print(mask)
print(array[mask])
print(array[array %2 == 0])
array[mask] = 0
print(array)

# Línea 1: comentario descriptivo.
# Línea 2: crea un array 1D de -3 a 3.
# Línea 3: lo imprime.
# Línea 4: crea una máscara booleana: True donde el elemento es par, False si
↳ es impar.
# Línea 5: imprime la máscara.
# Línea 6: array[mask] devuelve solo los elementos donde la máscara es True
↳ (los pares).
# Línea 7: hace lo mismo pero construyendo la condición directamente dentro de
↳ los corchetes.
# Línea 8: usa la máscara para asignar 0 a todas las posiciones pares (modifica
↳ el array in place).
# Línea 9: imprime el array final.
# Resultado esperado:
# [-3 -2 -1 0 1 2 3]
# [False True False True False True False]
# [-2 0 2]
# [-2 0 2]
# [-3 0 -1 0 1 0 3]

```

```

[-3 -2 -1 0 1 2 3]
[False True False True False True False]
[-2 0 2]

```

```
[-2  0  2]
[-3  0 -1  0  1  0  3]
```

```
[39]: array = np.arange(-3,4)
      print(array)
      array[array < 0] = 0
      print(array)

# Línea 1: crea un array 1D de -3 a 3.
# Línea 2: lo imprime.
# Línea 3: usa una condición booleana directamente como índice para poner en 0
    ↪ todos los
#           elementos negativos.
# Línea 4: imprime el resultado.
# Resultado esperado:
# [-3 -2 -1  0  1  2  3]
# [0  0  0  0  1  2  3]
```

```
[-3 -2 -1  0  1  2  3]
[0  0  0  0  1  2  3]
```

```
[40]: # slicing condicional
      array = np.arange(-3,4)
      print(array)
      mask = array % 2 == 0
      print(mask)
      array_par = array[mask] # devuelve una copia
      print(array_par)
      array_par[0] = -10
      print(array)
      print(array_par)

# Línea 1: comentario descriptivo.
# Línea 2: crea un array 1D de -3 a 3.
# Línea 3: lo imprime.
# Línea 4: crea la máscara booleana de elementos pares.
# Línea 5: imprime la máscara.
# Línea 6: a diferencia del slicing por rango, indexar con una máscara booleana
    ↪ (fancy indexing)
#           SÍ devuelve una copia independiente, no una vista.
# Línea 7: imprime la copia (los elementos pares).
# Línea 8: modifica el primer elemento de la copia.
# Línea 9: imprime el array original (sin cambios, porque array_par es una
    ↪ copia).
# Línea 10: imprime la copia modificada.
# Resultado esperado:
# [-3 -2 -1  0  1  2  3]
# [False  True False  True False  True False]
```

```
# [-2  0  2]
# [-3 -2 -1  0  1  2  3]
# [-10  0  2]
```

```
[-3 -2 -1  0  1  2  3]
[False  True False  True False  True False]
[-2  0  2]
[-3 -2 -1  0  1  2  3]
[-10  0  2]
```

- Acceso a múltiples valores con listados de índices

```
[41]: array = np.arange(0,16).reshape(4,4)
print(array)
print(array[[1,2]]) #selecting rows
print(array[:,[1,2]]) #selecting columns
# cross product of indexes
print(array[[1,2],:][:,[1,2]])
print(array[np.ix_([1,2],[1,2])]) # built in ix_
ar = array[[2,3,1]] # slice, copia
print(ar)
ar[0] = 999
print(ar)
print(array)

# Línea 1: crea la matriz 4x4 de 0 a 15.
# Línea 2: la imprime.
# Línea 3: array[[1,2]] selecciona las filas de índices 1 y 2 (fancy indexing,
↳ de filas).
# Línea 4: array[:,[1,2]] selecciona todas las filas pero solo las columnas de
↳ índices 1 y 2.
# Línea 5: comentario indicando que se busca el producto cruzado de índices
↳ (filas 1,2 con
#           columnas 1,2), no una selección elemento a elemento.
# Línea 6: primero selecciona filas [1,2] y luego, sobre ese resultado,
↳ columnas [1,2],
#           obteniendo la submatriz 2x2 de la intersección de esas filas y
↳ columnas.
# Línea 7: np.ix_([1,2],[1,2]) construye directamente los índices para lograr
↳ el mismo
#           producto cruzado en un solo paso.
# Línea 8: array[[2,3,1]] reordena/selecciona filas según la lista de índices
↳ dada (fila 2,
#           luego 3, luego 1); el resultado es una copia (fancy indexing), no
↳ una vista.
# Línea 9: imprime esa copia reordenada.
# Línea 10: modifica la primera fila de la copia 'ar'.
```

```

# Línea 11: imprime 'ar' modificada.
# Línea 12: imprime el array original 'array', que permanece SIN cambios porque
↳ ar es una copia.
# Resultado esperado: filas 1 y 2 ([[4,5,6,7],[8,9,10,11]]); columnas 1 y 2 de
↳ todas las filas;
#          submatriz 2x2 [[5,6],[9,10]] en ambos casos (loc y ix_); la copia
↳ reordenada con
#          filas [2,3,1]; esa misma copia con su primera fila en 999; y el
↳ array original
#          intacto (0..15).

```

```

[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]]
[[ 4  5  6  7]
 [ 8  9 10 11]]
[[ 1  2]
 [ 5  6]
 [ 9 10]
 [13 14]]
[[ 5  6]
 [ 9 10]]
[[ 5  6]
 [ 9 10]]
[[ 8  9 10 11]
 [12 13 14 15]
 [ 4  5  6  7]]
[[999 999 999 999]
 [ 12  13  14  15]
 [  4   5   6   7]]
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]]

```

1.4 3. Funciones matemáticas

- Aplicar operaciones entre escalares y arrays o entre arrays.

```

[42]: # listas y escalares
lista = [0,1,2,3]
lista *= 3
print(lista)
lista += [1]
print(lista)

# Línea 1: comentario descriptivo.

```

```

# Línea 2: se crea una lista Python de 4 elementos.
# Línea 3: en listas, el operador *= NO multiplica cada elemento; REPITE la
↳ lista 3 veces.
# Línea 4: imprime la lista repetida.
# Línea 5: += en listas concatena otra lista al final (añade el elemento 1).
# Línea 6: imprime el resultado final.
# Resultado esperado:
# [0, 1, 2, 3, 0, 1, 2, 3, 0, 1, 2, 3]
# [0, 1, 2, 3, 0, 1, 2, 3, 0, 1, 2, 3, 1]

```

```

[0, 1, 2, 3, 0, 1, 2, 3, 0, 1, 2, 3]
[0, 1, 2, 3, 0, 1, 2, 3, 0, 1, 2, 3, 1]

```

```

[43]: # ndarray hace las operaciones sobre los elementos
array = np.ones(4, dtype = np.int8)
print(array)
array *= 2
print(array)
array += 10
print(array)
print(array.dtype) # check data type of array

# Línea 1: comentario descriptivo.
# Línea 2: crea un array 1D de 4 unos con tipo de dato entero de 8 bits (int8).
# Línea 3: lo imprime.
# Línea 4: a diferencia de las listas, *= en un ndarray multiplica ELEMENTO A
↳ ELEMENTO (operación
#           vectorizada), no repite el array.
# Línea 5: imprime el array multiplicado por 2 ([2,2,2,2]).
# Línea 6: += suma 10 a cada elemento.
# Línea 7: imprime el array resultante ([12,12,12,12]).
# Línea 8: dtype muestra el tipo de dato interno del array (int8).
# Resultado esperado:
# [1 1 1 1]
# [2 2 2 2]
# [12 12 12 12]
# int8

```

```

[1 1 1 1]
[2 2 2 2]
[12 12 12 12]
int8

```

```

[44]: # operaciones entre arrays es como operaciones entre matrices
array_1 = np.ones(4) * 2
array_2 = np.arange(4) + 1
print(array_1)
print(array_2)

```



```

print(array_1 + array_2)
print(array_1 - array_2)
print(array_1 * array_2)
print(array_2 / array_1)

# Línea 1: comentario descriptivo.
# Línea 2: crea un array de 4 unos multiplicado por 2 -> [2. 2. 2. 2.]
# Línea 3: crea un array de 0 a 3 sumándole 1 -> [1 2 3 4]
# Línea 4-5: imprime ambos arrays.
# Línea 6: suma elemento a elemento -> [3. 4. 5. 6.]
# Línea 7: resta elemento a elemento -> [1. 0. -1. -2.]
# Línea 8: multiplicación elemento a elemento -> [2. 4. 6. 8.]
# Línea 9: división elemento a elemento (array_2 / array_1) -> [0.5 1. 1.5 2.]
# Resultado esperado: las seis impresiones descritas arriba, en ese orden.

```

```

[2. 2. 2. 2.]
[1 2 3 4]
[3. 4. 5. 6.]
[ 1.  0. -1. -2.]
[2. 4. 6. 8.]
[0.5 1.  1.5 2. ]

```

1.4.1 3.1 Broadcasting

Si las arrays no tienen las mismas dimensiones, se hace broadcast del pequeño al grande -> broadcasting

1.4.2 3.2 Operadores unarios y binarios

Tabla de referencia con operadores como `abs`, `sqrt`, `exp`, `log`, `sign`, `ceil`, `floor`, `isnan`, funciones trigonométricas (unarios) y `add`, `subtract`, `multiply`, `divide`, `maximum`, `minimum`, `equal`, `greater`, etc. (binarios).

```

[45]: # ejemplos de operadores
array = np.arange(5)
print(array)
np.sqrt(array)

# Línea 1: comentario descriptivo.
# Línea 2: crea un array 1D de 0 a 4.
# Línea 3: lo imprime.
# Línea 4: np.sqrt calcula la raíz cuadrada de cada elemento; al ser la última
    ↪ expresión de la
#           celda, Jupyter la muestra automáticamente como salida (sin necesidad
    ↪ de print).
# Resultado esperado:
# [0 1 2 3 4]
# array([0.          , 1.          , 1.41421356, 1.73205081, 2.          ])

```

```
[0 1 2 3 4]
```

```
[45]: array([0.          , 1.          , 1.41421356, 1.73205081, 2.          ])
```

```
[46]: # ejemplos de operadores
array1 = np.arange(4)
array2 = np.array([0,-1,2,-3])
print(array1)
print(array2)
print(np.greater(array1,array2))
print(np.less(array2, array1))

# Línea 1: comentario descriptivo.
# Línea 2: crea array1 = [0 1 2 3].
# Línea 3: crea array2 = [0 -1 2 -3].
# Línea 4-5: imprime ambos arrays.
# Línea 6: np.greater compara elemento a elemento si array1 > array2.
# Línea 7: np.less compara elemento a elemento si array2 < array1 (equivalente
    ↪ al resultado anterior).
# Resultado esperado:
# [0 1 2 3]
# [ 0 -1  2 -3]
# [False  True False  True]
# [False  True False  True]
```

```
[0 1 2 3]
[ 0 -1  2 -3]
[False  True False  True]
[False  True False  True]
```

1.5 4. Estadística descriptiva

- Importante saber la naturaleza de los datos.
- Valores máximos, mínimos, distribución, etc.

Funciones: sum, mean, std, cumsum, cumprod, min/max, any, all, unique, in1d, union1d, intersect1d.

```
[47]: # sum != cumsum
array1 = np.arange(9)
print(array1)
print(array1.sum())
print(array1.cumsum())

# Línea 1: comentario descriptivo.
# Línea 2: crea un array 1D de 0 a 8.
# Línea 3: lo imprime.
# Línea 4: sum() suma todos los elementos y devuelve un único escalar (0+1+...
    ↪ +8=36).
```

```
# Línea 5: cumsum() devuelve un array del mismo tamaño con la suma acumulada
↳progresiva.
# Resultado esperado:
# [0 1 2 3 4 5 6 7 8]
# 36
# [ 0  1  3  6 10 15 21 28 36]
```

```
[0 1 2 3 4 5 6 7 8]
36
[ 0  1  3  6 10 15 21 28 36]
```

```
[48]: # any vs all
array1 = np.arange(-8,2).reshape(2,5)
print(array1)
print(array1.any()) # any para saber si hay elementos True o valores <> 0
print(array1.all()) # all para saber si TODOS los elementos son True o valores
↳<> 0

# Línea 1: comentario descriptivo.
# Línea 2: crea un array de -8 a 1 (10 valores) reorganizado en 2x5.
# Línea 3: lo imprime.
# Línea 4: any() devuelve True si AL MENOS UN elemento es distinto de 0 (aquí
↳hay varios, así
#
que es True).
# Línea 5: all() devuelve True solo si TODOS los elementos son distintos de 0;
↳como el array
#
contiene un 0, el resultado es False.
# Resultado esperado:
# [[-8 -7 -6 -5 -4]
# [-3 -2 -1  0  1]]
# True
# False
```

```
[[ -8 -7 -6 -5 -4]
 [-3 -2 -1  0  1]]
True
False
```

```
[49]: # todas las funciones aceptan parametro 'axis'
# 0 para columnas, 1 para filas, 2 para profundidad...
array1 = np.arange(10).reshape(2,5)
print(array1)
print(array1.all(axis=0))
print(array1.all(axis=1))
print(array1.all())

# Línea 1-2: comentarios descriptivos sobre el parámetro axis.
# Línea 3: crea un array de 0 a 9 reorganizado en 2x5.
```

```

# Línea 4: lo imprime.
# Línea 5: all(axis=0) evalúa 'all' por columna (recorriendo filas): para cada
    ↪ una de las 5
#
    columnas revisa si todos sus valores son distintos de 0.
# Línea 6: all(axis=1) evalúa 'all' por fila (recorriendo columnas): para cada
    ↪ una de las 2 filas.
# Línea 7: all() sin axis evalúa sobre todo el array (será False porque
    ↪ contiene un 0).
# Resultado esperado:
# [[0 1 2 3 4]
#  [5 6 7 8 9]]
# [False True True True True]
# [False True]
# False

```

```

[[0 1 2 3 4]
 [5 6 7 8 9]]
[False True True True True]
[False True]
False

```

1.6 5. Álgebra lineal

- `numpy.linalg` contiene funciones para álgebra lineal: producto punto, multiplicación de matrices, determinante, factorizaciones, etc.

```

[50]: A = np.arange(12).reshape(4,3)
      B = np.arange(6).reshape(3,2)
      print(A)
      print(B)
      print(np.matmul(A, B))
      print(np.dot(A, B))
      print(A @ B)

# Línea 1: crea la matriz A de 4x3 con valores 0..11.
# Línea 2: crea la matriz B de 3x2 con valores 0..5.
# Línea 3-4: imprime ambas matrices.
# Línea 5: np.matmul realiza la multiplicación matricial estándar (4x3 · 3x2 =
    ↪ 4x2).
# Línea 6: np.dot, para matrices 2D, produce el mismo resultado que matmul.
# Línea 7: el operador @ es el operador de multiplicación matricial de Python
    ↪ (equivalente a
#
    matmul/dot para matrices 2D).
# Resultado esperado: A y B impresas, y tres veces la misma matriz resultado
    ↪ 4x2:
# [[ 10  13]
#  [ 28  40]

```

```
# [ 46  67]
# [ 64  94]]
```

```
[[ 0  1  2]
 [ 3  4  5]
 [ 6  7  8]
 [ 9 10 11]]
[[0 1]
 [2 3]
 [4 5]]
[[10 13]
 [28 40]
 [46 67]
 [64 94]]
[[10 13]
 [28 40]
 [46 67]
 [64 94]]
[[10 13]
 [28 40]
 [46 67]
 [64 94]]
```

```
[51]: import numpy.linalg as lg
import math
# resolver sistema de ecuaciones
#  $3x + 2y + 5z = 12$ 
#  $x - 3y - z = 2$ 
#  $2x - y + 12z = 32$ 
#  $Ax = b \rightarrow$  resolver para  $x$ 
a = np.array([ [3,2,5], [1,-3,-1], [2,-1,12] ])
b = np.array([12,2,32])
print(a)
print(b)
x,y,z = lg.solve(a,b)
print(f'x={x} y={y} z={z}')
print(math.isclose(3*x + 2*y + 5*z, 12))
print(math.isclose(x - 3*y - z, 2))
print(math.isclose(2*x - y + 12*z, 32))

# Línea 1: importa el submódulo de álgebra lineal de NumPy con alias 'lg'.
# Línea 2: importa el módulo estándar math (para comparar floats con
↳ tolerancia).
# Línea 3-7: comentarios que describen el sistema de ecuaciones lineales a
↳ resolver.
# Línea 8: matriz de coeficientes 'a' (3x3) del sistema  $Ax = b$ .
# Línea 9: vector de términos independientes 'b'.
```

```

# Línea 10-11: imprime 'a' y 'b'.
# Línea 12: lg.solve(a,b) resuelve el sistema lineal y desempaqueta la solución
↳ en x, y, z.
# Línea 13: imprime los valores calculados de x, y, z.
# Línea 14-16: math.isclose verifica (con tolerancia de punto flotante) que
↳ cada ecuación
#
original se cumple sustituyendo los valores hallados.
# Resultado esperado: se imprime la matriz 'a', el vector 'b', los valores de
↳ x, y, z que
#
resuelven el sistema, y tres 'True' confirmando que las tres
↳ ecuaciones se satisfacen.

```

```

[[ 3  2  5]
 [ 1 -3 -1]
 [ 2 -1 12]]
[12  2 32]
x=0.7543859649122807 y=-1.2280701754385965 z=2.43859649122807
True
True
True

```

```

[52]: #matriz identidad
I = np.identity(4)
print(I)
A = np.arange(16).reshape(4,4)
print(A)
print(np.matmul(A,I))

# Línea 1: comentario descriptivo.
# Línea 2: np.identity(4) crea la matriz identidad 4x4.
# Línea 3: la imprime.
# Línea 4: crea la matriz A de 4x4 con valores 0..15.
# Línea 5: la imprime.
# Línea 6: multiplicar cualquier matriz por la identidad da como resultado la
↳ misma matriz
#
(propiedad del elemento neutro de la multiplicación matricial), pero
↳ convertida a float.
# Resultado esperado: matriz identidad 4x4, matriz A original, y A de nuevo (en
↳ formato float)
#
como resultado de A @ I.

```

```

[[1.  0.  0.  0.]
 [0.  1.  0.  0.]
 [0.  0.  1.  0.]
 [0.  0.  0.  1.]]
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]

```

```
[12 13 14 15]]
[[ 0.  1.  2.  3.]
 [ 4.  5.  6.  7.]
 [ 8.  9. 10. 11.]
[12. 13. 14. 15.]]
```

1.7 6. Filtrado de datos

- Filtrar y modificar datos numéricos con `np.where`.
- Retorna A o B en función de una condición en un array.

```
[53]: prices = np.array([0.99, 14.49, 19.99, 20.99, 0.49])
mask = prices < 1
print(prices[prices < 1]) #slicing condicional
# mask con los elementos menores de 1
mask = np.where(prices < 1, True, False)
print(mask)
print(prices[mask])
prices[mask] = 1.0
print(prices)
print(np.where(prices < 1, 1.0, prices))

# Línea 1: crea un array de precios.
# Línea 2: crea una máscara booleana de los precios menores a 1.
# Línea 3: imprime, con slicing condicional directo, los precios menores a 1.
# Línea 4: (descomentado) comentario que indica que a continuación se construye
↳ la máscara.
# Línea 5: (descomentado) np.where(condición, valor_si_true, valor_si_false)
↳ construye la misma
#
↳ máscara booleana pero explícitamente con np.where en vez del
↳ operador <.
# Línea 6: (descomentado) imprime la máscara booleana.
# Línea 7: (descomentado) imprime los precios seleccionados con la máscara (los
↳ menores a 1).
# Línea 8: (descomentado) asigna 1.0 a todas las posiciones marcadas por la
↳ máscara (redondeo
↳ hacia arriba de los precios más baratos).
# Línea 9: (descomentado) imprime el array de precios ya modificado.
# Línea 10: np.where también puede usarse para crear un NUEVO array: donde la
↳ condición es
↳ verdadera coloca 1.0, si no, conserva el valor original de 'prices'
↳ (que ya fue
↳ modificado en la línea anterior, así que aquí no quedan precios
↳ menores a 1).
# Resultado esperado:
# [0.99 0.49]
# [ True False False False  True]
```

```
# [0.99 0.49]
# [1.    14.49 19.99 20.99 1.   ]
# [1.    14.49 19.99 20.99 1.   ]
```

```
[0.99 0.49]
[ True False False False  True]
[0.99 0.49]
[ 1.    14.49 19.99 20.99 1.   ]
[ 1.    14.49 19.99 20.99 1.   ]
```

```
[54]: # Se puede usar para sustituir datos fuera de rango
bank_transfers_values = np.array([0.99, -1.49, 19.99, 20.99, -0.49, 12.1]).
    ↳ reshape(2, 3)
print(bank_transfers_values)
clean_data = np.where(bank_transfers_values > 0, bank_transfers_values, 0)
print(clean_data)

# Línea 1: comentario descriptivo.
# Línea 2: crea un array de 6 valores (algunos negativos) y lo reorganiza en
    ↳ 2x3.
# Línea 3: lo imprime.
# Línea 4: np.where reemplaza cada valor negativo (condición False) por 0, y
    ↳ conserva los
#           positivos (condición True) tal cual.
# Línea 5: imprime el array "limpio".
# Resultado esperado:
# [[ 0.99 -1.49 19.99]
#  [20.99 -0.49 12.1 ]]
# [[ 0.99  0.   19.99]
#  [20.99  0.   12.1 ]]

[[ 0.99 -1.49 19.99]
 [20.99 -0.49 12.1 ]]
[[ 0.99  0.   19.99]
 [20.99  0.   12.1 ]]
```

```
[55]: # limpiar NaN (sustitución)
data = [10, 12, -143, np.nan, 1, -3] # np.nan --> NotANumber, elemento especial
data_clean = np.where(np.isnan(data), 0, data)
print(data_clean)
print(id(data))
print(id(data_clean))

# Línea 1: comentario descriptivo.
# Línea 2: se define una lista Python con un valor especial np.nan (Not a
    ↳ Number).
# Línea 3: np.isnan(data) genera una máscara True donde el valor es NaN; np.
    ↳ where sustituye
```



```
#      esos NaN por 0 y conserva el resto de valores (data se convierte
↳ implícitamente en
#      ndarray).
# Línea 4: imprime el array limpio.
# Línea 5: id(data) muestra la dirección de memoria del objeto lista original.
# Línea 6: id(data_clean) muestra la dirección de memoria del nuevo ndarray
↳ (distinta a la de
#      'data', confirmando que np.where devuelve un objeto nuevo, no
↳ modifica in place).
# Resultado esperado:
# [ 10.  12. -143.   0.   1.  -3.]
# <id de 'data', un número entero>
# <id de 'data_clean', un número entero distinto>
```

```
[ 10.  12. -143.   0.   1.  -3.]
2135785170752
2135787144560
```

```
[56]: # np.where puede seleccionar elementos
bank_transfers_values = np.array([0.99, -1.49, 19.99, 20.99, -0.49, 12.1]).
↳ reshape(2, 3)
print(bank_transfers_values)
credits = np.where(bank_transfers_values > 0) # array de índices para los que
↳ la condición es True
print(credits)
print(bank_transfers_values[credits]) # es igual que
↳ bank_transfers_values[bank_transfers_values > 0]
print(bank_transfers_values[bank_transfers_values > 0])

# Línea 1: comentario descriptivo.
# Línea 2: crea la misma matriz 2x3 de valores con signo.
# Línea 3: la imprime.
# Línea 4: np.where con UN SOLO argumento (la condición) no sustituye valores,
↳ sino que devuelve
#      una tupla con los índices (por eje) donde la condición es verdadera.
# Línea 5: imprime esa tupla de índices (arrays de filas y columnas).
# Línea 6: usar esos índices para indexar el array original devuelve los
↳ valores positivos.
# Línea 7: forma equivalente más directa usando la condición booleana como
↳ índice.
# Resultado esperado:
# [[ 0.99 -1.49 19.99]
#  [ 20.99 -0.49 12.1 ]]
# (array([0, 0, 1, 1]), array([0, 2, 0, 2]))
# [ 0.99 19.99 20.99 12.1 ]
# [ 0.99 19.99 20.99 12.1 ]
```

```
[[ 0.99 -1.49 19.99]
```

```
[20.99 -0.49 12.1 ]]
(array([0, 0, 1, 1]), array([0, 2, 0, 2]))
[ 0.99 19.99 20.99 12.1 ]
[ 0.99 19.99 20.99 12.1 ]
```

```
[57]: # valor de array dependiendo del valor en array referencia
rewards_default = np.arange(6)
print(rewards_default)
rewards_upgrade = np.arange(6) * 10
print(rewards_upgrade)
daily_points = np.array([0, 0, 1, 6, 0, 2])
print(daily_points)
final_rewards = np.where(daily_points > 1, rewards_upgrade, rewards_default)
print(final_rewards)

# Línea 1: comentario descriptivo.
# Línea 2: crea el array de recompensas por defecto (0..5).
# Línea 3: lo imprime.
# Línea 4: crea el array de recompensas mejoradas (0,10,20,30,40,50).
# Línea 5: lo imprime.
# Línea 6: crea un array con los puntos diarios obtenidos por posición/día.
# Línea 7: lo imprime.
# Línea 8: np.where compara elemento a elemento: si daily_points > 1, toma el
    ↪ valor de
#         rewards_upgrade en esa posición; si no, toma el valor de
    ↪ rewards_default.
# Línea 9: imprime el array combinado resultante.
# Resultado esperado:
# [0 1 2 3 4 5]
# [ 0 10 20 30 40 50]
# [0 0 1 6 0 2]
# [ 0  1  2 30  4 20]
```

```
[0 1 2 3 4 5]
[ 0 10 20 30 40 50]
[0 0 1 6 0 2]
[ 0  1  2 30  4 50]
```

1.8 7. Números aleatorios

- Módulo estándar `random` de Python.

```
[58]: import random
print(random.random()) # número entre 0 y 1
print(random.randint(0,5)) # integral entre dos valores
print(random.uniform(0,5)) # real entre dos valores

# Línea 1: importa el módulo random de la librería estándar de Python.
```

```
# Línea 2: random.random() genera un float aleatorio en el rango [0.0, 1.0).
# Línea 3: random.randint(0,5) genera un entero aleatorio entre 0 y 5, AMBOS
↳inclusive.
# Línea 4: random.uniform(0,5) genera un float aleatorio entre 0 y 5
↳(distribución uniforme).
# Resultado esperado: tres valores aleatorios distintos en cada ejecución, por
↳ejemplo:
# 0.3745...
# 3
# 2.891...
```

0.2446205806441475

0

1.731470926491872

```
[59]: # elegir un elemento al azar dentro de una colección
names = ['Pikachu', 'Eevee', 'Charmander']
random.choice(names)

# Línea 1: comentario descriptivo.
# Línea 2: se define una lista de nombres.
# Línea 3: random.choice selecciona y devuelve UN elemento al azar de la lista
↳(con
#       distribución uniforme); al ser la última expresión de la celda,
↳Jupyter la muestra
#       como salida.
# Resultado esperado: uno de los tres strings, elegido aleatoriamente, p. ej.
↳'Eevee'.
```

[59]: 'Charmander'

1.8.1 7.1 NumPy.random

- Generar fácilmente listas con valores aleatorios.

```
[60]: print(np.random.rand()) # número entre 0 y 1
print(np.random.randint(0,5)) # integral entre dos valores
print(np.random.randint(5, size=(2, 4))) # parametro 'size' para indicar las
↳dimensiones

# Línea 1: np.random.rand() genera un float aleatorio uniforme en [0,1).
# Línea 2: np.random.randint(0,5) genera un entero aleatorio en [0,5) (5
↳EXCLUSIVO, a diferencia
#       de random.randint del módulo estándar).
# Línea 3: np.random.randint(5, size=(2,4)) genera una matriz 2x4 de enteros
↳aleatorios en [0,5).
```

```
# Resultado esperado: un float aleatorio, un entero aleatorio entre 0 y 4, y
↪ una matriz 2x4 de
# enteros aleatorios entre 0 y 4 (valores distintos en cada ejecución).
```

```
0.22093144413424493
```

```
1
```

```
[[0 2 1 3]
 [4 1 3 1]]
```

```
[61]: # array de elementos aleatorios (distribucion normal gaussiana, media 0,
↪ desviación 1)
print(np.random.randn(4,2))

# Línea 1: comentario descriptivo.
# Línea 2: np.random.randn(4,2) genera una matriz 4x2 de números aleatorios
↪ muestreados de una
# distribución normal estándar (media 0, desviación estándar 1).
# Resultado esperado: una matriz 4x2 de valores float, mayormente entre -3 y 3,
↪ distintos en
# cada ejecución.
```

```
[[-0.06039481 -0.87249768]
 [-0.76142463  2.22902307]
 [ 0.20680328  0.28387643]
 [ 1.62932497  0.08758933]]
```

```
[62]: # otras distribuciones
print(np.random.binomial(n=5, p=0.3)) # 'n' intentos, 'p' probabilidad
print(np.random.uniform(low=0, high=10)) # distribución uniforme
print(np.random.poisson(lam=2, size=(2,2))) # 'lam' número de ocurrencias
↪ esperadas, retornar 'size' valores

# Línea 1: comentario descriptivo.
# Línea 2: genera un entero aleatorio de una distribución binomial con 5
↪ intentos y probabilidad
# de éxito 0.3 (número de éxitos entre 0 y 5).
# Línea 3: genera un float aleatorio uniforme entre 0 y 10.
# Línea 4: genera una matriz 2x2 de enteros aleatorios muestreados de una
↪ distribución de
# Poisson con media esperada (lam) de 2 ocurrencias.
# Resultado esperado: un entero entre 0 y 5, un float entre 0 y 10, y una
↪ matriz 2x2 de enteros
# no negativos (todos aleatorios, distintos en cada ejecución).
```

```
1
```

```
0.7740462980017404
```

```
[[2 0]
 [3 1]]
```

1.8.2 7.2 Random seed

- Números pseudo-aleatorios.
- Los ordenadores generan números a partir de ecuaciones.
- Basadas en un número inicial (seed).
- Bueno para reproducibilidad de experimentos.

```
[63]: i = 5
for _ in range(5):
    np.random.seed(i) # misma semilla
    print(np.random.rand())
    print(np.random.rand())
print()

for i in range(6):
    np.random.seed(i) # diferentes semillas
    print(np.random.rand())
    print(np.random.rand())

# Línea 1: fija la variable i en 5.
# Línea 2: bucle que se repite 5 veces (el valor de la variable de iteración se
↳ descarta con '_').
# Línea 3: en cada iteración se reinicia la semilla del generador aleatorio
↳ siempre al mismo
#         valor (i=5).
# Línea 4-5: al usar SIEMPRE la misma semilla, los dos números aleatorios
↳ generados con
#         np.random.rand() serán IDÉNTICOS en cada una de las 5 iteraciones
↳ del bucle.
# Línea 6: imprime una línea en blanco para separar los dos bloques.
# Línea 7: bucle donde i toma los valores 0,1,2,3,4,5.
# Línea 8: en cada iteración se usa una semilla DIFERENTE (el propio valor de
↳ i).
# Línea 9-10: al cambiar la semilla en cada vuelta, los pares de números
↳ generados serán
#         distintos entre sí, pero siempre reproducibles si se repite la
↳ ejecución completa.
# Resultado esperado: primero 5 pares de números IDÉNTICOS entre sí (misma
↳ semilla=5), luego
#         una línea en blanco, y después 6 pares de números diferentes entre
↳ sí (uno por cada
#         semilla 0..5), pero siempre los mismos valores si se vuelve a
↳ ejecutar la celda.
```

```
0.22199317108973948
0.8707323061773764
0.22199317108973948
0.8707323061773764
```

```

0.22199317108973948
0.8707323061773764
0.22199317108973948
0.8707323061773764
0.22199317108973948
0.8707323061773764

0.5488135039273248
0.7151893663724195
0.417022004702574
0.7203244934421581
0.43599490214200376
0.025926231827891333
0.5507979025745755
0.7081478226181048
0.9670298390136767
0.5472322491757223
0.22199317108973948
0.8707323061773764

```

1.9 8. Escritura y lectura de ndarrays a archivos

- Para futuro acceso.
- Compartir datos con otros.
- Formato *.npv.

```

[64]: # escribir a archivo
import os
ruta = os.path.join("res" ,"o_values_array.npy")
values = np.random.randint(9, size=(3,3))
np.save(ruta, values)
print(values)

# Línea 1: comentario descriptivo.
# Línea 2: importa el módulo os para construir rutas de forma independiente del
↪ sistema operativo.
# Línea 3: construye la ruta 'res/o_values_array.npy'.
# Línea 4: genera una matriz 3x3 de enteros aleatorios entre 0 y 8.
# Línea 5: np.save guarda el array en disco en formato binario .npv en la ruta
↪ indicada
#           (requiere que la carpeta 'res' ya exista).
# Línea 6: imprime la matriz generada.
# Resultado esperado: se crea el archivo 'res/o_values_array.npy' en disco y se
↪ imprime la
#           matriz 3x3 de enteros aleatorios generada.

```

```

[[6 6 0]
 [8 4 7]

```

```
[0 0 7]]
```

```
[65]: # leer de archivo
old_values = np.load(ruta)
print(old_values)

# Línea 1: comentario descriptivo.
# Línea 2: np.load lee el array binario previamente guardado en 'ruta' y lo
↳reconstruye en memoria.
# Línea 3: imprime el array cargado.
# Resultado esperado: la misma matriz 3x3 que se guardó en la celda anterior.
```

```
[[6 6 0]
 [8 4 7]
 [0 0 7]]
```

```
[66]: # escribir y cargar múltiples arrays
ruta = os.path.join("res" , "o_array_group.npz")
values1 = np.random.randint(9,size=(3,3))
values2 = np.random.randint(4,size=(2,2))
values3 = np.random.randint(16,size=(4,4))
np.savez(ruta,values1,values2,values3) # añade la extensión .npz

# Línea 1: comentario descriptivo.
# Línea 2: construye la ruta 'res/o_array_group.npz'.
# Línea 3-5: genera tres arrays aleatorios de tamaños distintos.
# Línea 6: np.savez guarda VARIOS arrays comprimidos en un único archivo .npz
↳(cada array queda
#
almacenado con una clave automática arr_0, arr_1, arr_2).
# Resultado esperado: no hay salida impresa; se crea el archivo 'res/
↳o_array_group.npz' con los
#
tres arrays dentro.
```

```
[67]: # leer los archivos (retorna un diccionario con las arrays)
npzfile = np.load(ruta)
print(type(npzfile.files))
print(npzfile.files)
for key in npzfile.files: # array.files contiene el nombre de las arrays
↳(arr_0, arr_1...)
print(npzfile[key])

# Línea 1: comentario descriptivo.
# Línea 2: np.load sobre un .npz devuelve un objeto tipo NpzFile que se
↳comporta como un
#
diccionario de arrays.
# Línea 3: muestra el tipo de npzfile.files (una lista de nombres de claves).
# Línea 4: imprime esa lista de claves (nombres autogenerados de los arrays
↳guardados).
```

```
# Línea 5: itera sobre cada clave del archivo.
# Línea 6: para cada clave, imprime el array correspondiente accediendo como si
↳ fuera un diccionario.
# Resultado esperado: el tipo <class 'list'>, la lista
↳ ['arr_0', 'arr_1', 'arr_2'], y a
#
    continuación se imprimen los tres arrays guardados (3x3, 2x2 y 4x4).
```

```
<class 'list'>
['arr_0', 'arr_1', 'arr_2']
[[1 5 7]
 [0 1 4]
 [6 2 1]]
[[2 3]
 [0 2]]
[[ 5 10  0  0]
 [ 4 14  4  9]
 [ 3 11 15  2]
 [ 4  6  9  3]]
```

¿Por qué molestarse con .npz?

- Bases de datos suelen estar en CSV o txt, que requieren lectura con streams (parsing).
- ¿Para qué escribir los datos de nuevo?

```
[ ]: import os
ruta_csv = os.path.join("res" , "fdata.csv")
with open(ruta_csv, 'r') as f:
    data_str = f.read()
data = data_str.split(',')
data_array = np.array(data, dtype = np.int8).reshape(1000,1000)
# print(data_array)
ruta_npy = os.path.join("res" , "o_fdata.npy")
np.save(ruta_npy, data_array)

# Línea 1: importa os.
# Línea 2: construye la ruta del archivo CSV de origen ('res/fdata.csv').
# Línea 3: abre el archivo CSV en modo lectura ('r') usando un context manager
↳ (with), que
#
    garantiza que el archivo se cierre automáticamente al salir del
↳ bloque.
# Línea 4: lee todo el contenido del archivo como un único string.
# Línea 5: separa el string por comas, obteniendo una lista de strings
↳ numéricos.
# Línea 6: convierte esa lista en un ndarray de tipo int8 y la reorganiza en
↳ una matriz 1000x1000.
# Línea 7: (comentado intencionalmente en el original) evita imprimir un millón
↳ de valores en pantalla.
```



```
# Línea 8: construye la ruta de salida 'res/o_fdata.npy'.
# Línea 9: guarda el array ya parseado en formato binario .npy para
↳ reutilizarlo sin tener que
# volver a parsear el CSV cada vez.
# Resultado esperado: no hay salida impresa (el print está comentado); se crea
↳ el archivo
# 'res/o_fdata.npy' con la matriz 1000x1000 lista para cargarse
↳ rápidamente.
```

```
[ ]: %%timeit # magic function jupyter
with open(ruta_csv, 'r') as f:
    data_str = f.read()
data = data_str.split(',')
data_array = np.array(data, dtype = np.int8).reshape(1000,1000)

# Línea 1: %%timeit es una magic function de Jupyter que ejecuta el contenido
↳ de la celda
# múltiples veces y reporta el tiempo promedio de ejecución.
# Línea 2-5: se repite el proceso completo de abrir, leer, parsear (split) y
↳ convertir el CSV
# en un ndarray de 1000x1000, tal como en la celda anterior.
# Resultado esperado: un reporte de tiempo promedio (p. ej. "12.5 ms ± 300 μs
↳ per loop"),
# representando el costo de parsear el CSV cada vez desde texto plano.
```

```
[ ]: %%timeit
data = np.load(ruta_npy)

# Línea 1: %%timeit mide el tiempo promedio de ejecución de la celda.
# Línea 2: np.load carga directamente el array binario ya preprocesado desde el
↳ archivo .npy.
# Resultado esperado: un tiempo de ejecución MUCHO menor que el del parseo del
↳ CSV (típicamente
# microsegundos frente a milisegundos), demostrando la ventaja de usar
↳ formato .npy
# para datos ya procesados.
```

```
[ ]: %%timeit # magic function jupyter
result = np.fromfile(ruta_csv, dtype=np.int8, sep=",")

# Línea 1: %%timeit mide el tiempo promedio de ejecución.
# Línea 2: np.fromfile con sep="," lee el CSV directamente como texto
↳ delimitado por comas y lo
# convierte a un ndarray de tipo int8 en un solo paso, sin pasar por
↳ un string intermedio
# completo ni por split() manual.
```

```
# Resultado esperado: un tiempo de ejecución intermedio entre el parseo manual
↳ con split() y la
# carga directa del .npy (más rápido que split() pero más lento que np.
↳ load), mostrando
# una alternativa más directa de lectura de CSV con NumPy.
```

Velocidad + simplicidad vs memoria

1.9.1 8.1 Ejercicios

- Ejercicios para practicar NumPy: https://github.com/rougier/numpy100/blob/master/100_Numpy_exercises